

HOMEWORK 5

>>Matej Popovski<<
>>9083541632<<

Instructions:

Use this LaTeX file as a template to develop your homework and submit it as a single PDF file on Gradescope. Remember to assign the pages to problems when submitting.

1 Optimal Policy for Simple MDP

Consider the simple n -state MDP shown in Figure 1. Starting from state s_1 , the agent can move to the right (a_0) or left (a_1) from any state s_i . Actions are deterministic and always succeed (e.g. going left from state s_2 goes to state s_1 , and going left from state s_1 transitions to itself). Rewards are given upon taking an action from the state. Taking any action from the goal state G earns a reward of $r = +1$ and the agent stays in state G . Otherwise, each move has zero reward ($r = 0$). Assume a discount factor $\gamma < 1$.

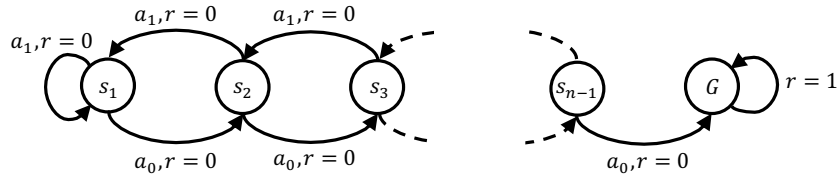


Figure 1: n -state MDP

1.1 Optimal action [5 points]

What is the optimal policy, and does it depend on the value of the discount factor γ ? Explain your answer.

The only positive reward in this MDP is obtained in the goal state G , where taking any action gives reward $r = 1$ and leaves the agent in G . From any non-goal state s_i the two actions behave as follows: action a_0 moves one step to the right (toward G), while action a_1 moves one step to the left (away from G , and from s_1 it keeps the agent in s_1). Since rewards outside G are 0 and the discount factor satisfies $\gamma < 1$, it is always better to reach G in as few steps as possible.

Therefore, the optimal policy π^* is

$$\pi^*(s_i) = a_0 \quad \text{for all } i = 1, \dots, n-1,$$

i.e., always move right until reaching G ; in G any action is optimal since both actions give the same reward and keep the agent in G .

This policy does *not* depend on the particular value of γ (as long as $0 \leq \gamma < 1$), because any delay in reaching G only postpones the positive rewards and makes them less valuable under discounting.

1.2 Optimal value function [10 points]

Find the optimal value function for all states s_i and the goal state G .

Let $V^*(s)$ denote the optimal value function.

For the goal state G , taking any action yields reward 1 and leaves the agent in G . Thus

$$V^*(G) = 1 + \gamma V^*(G),$$

which gives

$$V^*(G) = \frac{1}{1 - \gamma}.$$

For a non-goal state s_i ($i = 1, \dots, n-1$), under the optimal policy we always take action a_0 (move right). This gives reward 0 and moves to s_{i+1} , so

$$V^*(s_i) = 0 + \gamma V^*(s_{i+1}) = \gamma V^*(s_{i+1}).$$

Applying this recursively until reaching G and noting that the number of steps from s_i to G is $d_i = n - i$, we obtain

$$V^*(s_i) = \gamma^{d_i} V^*(G) = \gamma^{n-i} \frac{1}{1-\gamma}.$$

In summary,

$$V^*(G) = \frac{1}{1-\gamma} \quad \text{and} \quad V^*(s_i) = \frac{\gamma^{n-i}}{1-\gamma} \quad \text{for } i = 1, \dots, n-1.$$

1.3 Adding a constant to each reward [20 points]

Consider adding a constant c to all rewards (i.e. taking any action from states s_i has reward c and any action from the goal state G has reward $1 + c$). Can adding a constant reward c change the optimal policy? Derive the new optimal value function for all states s_i and the goal state G . Explain your answer.

Now we add a constant c to every reward: each transition from a non-goal state yields reward c , while each transition in G yields reward $1 + c$.

Adding the same constant c to every reward at every time step shifts the return of *any* trajectory by the same constant

$$\frac{c}{1-\gamma},$$

so the relative ordering of policies does not change. Therefore, the optimal policy remains the same as in 1.1 (always move right until reaching G).

To find the new value function, first consider G :

$$V_c(G) = (1 + c) + \gamma V_c(G) \Rightarrow V_c(G) = \frac{1 + c}{1 - \gamma}.$$

From a non-goal state s_i , we are $d_i = n - i$ steps away from G under the optimal policy. For the first d_i steps we receive reward c , and thereafter we receive $(1 + c)$ at each step in G . Thus

$$V_c(s_i) = c \sum_{k=0}^{d_i-1} \gamma^k + \gamma^{d_i} \frac{1 + c}{1 - \gamma} = \frac{c(1 - \gamma^{d_i}) + \gamma^{d_i}(1 + c)}{1 - \gamma} = \frac{c + \gamma^{d_i}}{1 - \gamma}.$$

Substituting $d_i = n - i$ gives

$$V_c(s_i) = \frac{c + \gamma^{n-i}}{1 - \gamma}, \quad i = 1, \dots, n-1.$$

Note that $V_c(s) = V^*(s) + \frac{c}{1-\gamma}$ for every state s , i.e., the value function is shifted by a constant but the optimal policy is unchanged.

1.4 Scaling all the rewards by a constant [15 points]

After adding a constant c to all rewards now consider scaling all the rewards by a constant a (i.e. $r_{new} = a(c + r_{old})$). Can scaling rewards by a change the optimal policy? Derive the new optimal value function for all states s_i and the goal state G .

Now we scale all rewards from part 1.3 by a constant factor a . That is, every reward r is replaced by ar . For any trajectory with original return G , the new return is simply aG .

If $a > 0$, scaling all returns by a positive constant preserves the ordering over policies, so the optimal policy is again unchanged (we still always move right until reaching G).

Because every reward is multiplied by a , the optimal value function is also scaled by a . Hence

$$V_{a,c}(s) = a V_c(s) \quad \text{for all states } s.$$

Using the expressions from 1.3, we obtain

$$V_{a,c}(G) = a \frac{1 + c}{1 - \gamma},$$

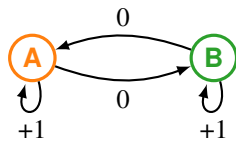
and for $i = 1, \dots, n - 1$,

$$V_{a,c}(s_i) = a \frac{c + \gamma^{n-i}}{1 - \gamma}.$$

(We assume $a > 0$. If $a = 0$, all rewards become zero and every policy has value 0, so the problem becomes degenerate. If $a < 0$, multiplying all rewards by a negative constant reverses the ordering over policies, so the optimal policy would be the one that *minimizes* the original return. These cases are not considered in this problem.)

2 Q-learning

Consider the following Markov Decision Process. It has two states s . It has two actions a : move and stay. The state transition is deterministic: “move” moves to the other state, while “stay” stays at the current state. The reward r is 0 for move, 1 for stay. There is a discounting factor $\gamma = 0.8$.



The reinforcement learning agent performs Q-learning. Recall the Q table has entries $Q(s, a)$. The Q table is initialized with all zeros. The agent starts in state $s_1 = A$. In any state s_t , the agent chooses the action a_t according to a behavior policy $a_t = \pi_B(s_t)$. Upon experiencing the next state and reward s_{t+1}, r_t the update is:

$$Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha \left(r_t + \gamma \max_{a'} Q(s_{t+1}, a') \right).$$

Let the step size parameter $\alpha = 0.5$.

2.1 Greedy [15 points]

Run Q-learning for 200 steps with a deterministic greedy behavior policy: at each state s_t use the best action $a_t \in \arg \max_a Q(s_t, a)$ indicated by the current Q table. If there is a tie, prefer move. Show the Q table at the end.

We have two states A and B , and two actions in each state: move (switch to the other state) and stay (remain in the same state). The reward is 0 for move and 1 for stay. The discount factor is $\gamma = 0.8$, the learning rate is $\alpha = 0.5$, and we initialize

$$Q(A, \text{move}) = Q(A, \text{stay}) = Q(B, \text{move}) = Q(B, \text{stay}) = 0.$$

Under a greedy policy, at each step we choose the action with the highest $Q(s, \cdot)$. In case of a tie, we are told to prefer move. Initially, in both states A and B the two actions are tied at 0, so the agent always chooses move in both states and thus alternates deterministically between A and B :

$$A \xrightarrow{\text{move}} B \xrightarrow{\text{move}} A \xrightarrow{\text{move}} B \dots$$

Because stay is never selected, its Q -values are never updated and remain 0. For move, every transition gives immediate reward 0 and goes to a state whose Q -values are also all 0. The Q-learning update for move therefore becomes

$$Q_{t+1}(s, \text{move}) = Q_t(s, \text{move}) + \alpha [0 + \gamma \max_{a'} Q_t(s', a') - Q_t(s, \text{move})] = Q_t(s, \text{move}),$$

since both $Q_t(s, \text{move})$ and $\max_{a'} Q_t(s', a')$ are 0 at all times. Thus $Q(s, \text{move})$ also stays at 0. After 200 steps (indeed, after any number of steps), the Q -table is

$$Q(A, \text{move}) = Q(A, \text{stay}) = 0, \quad Q(B, \text{move}) = Q(B, \text{stay}) = 0.$$

This illustrates that a purely greedy policy can fail to learn the value of better actions when it never explores them.

2.2 ϵ -greedy [15 points]

Reset and repeat the above, but with an ϵ -greedy behavior policy: at each state s_t , with probability $1 - \epsilon$ choose what the current Q table says is the best action: $\arg \max_a Q(s_t, a)$; Break ties arbitrarily. Otherwise, (with probability ϵ) uniformly chooses between move and stay (move or stay both with 1/2 probability). Use $\epsilon = 0.5$. Now we use an ϵ -greedy policy with $\epsilon = 0.5$. At each step:

- with probability $1 - \epsilon = 0.5$ we choose the greedy action (maximizing $Q(s, \cdot)$), and
- with probability $\epsilon = 0.5$ we choose an action uniformly at random from $\{\text{move}, \text{stay}\}$.

Because every action in every state has a nonzero probability of being chosen, both `move` and `stay` are visited infinitely often. In particular, the agent sometimes chooses `stay`, receives reward 1, and updates $Q(s, \text{stay})$ upward. Over time, $Q(s, \text{stay})$ becomes larger than $Q(s, \text{move})$, so the greedy part of the policy will increasingly prefer `stay`.

In the limit (and approximately after many updates, such as 200 steps), Q-learning converges to the optimal action-values Q^* . We can compute these directly. In either state (*A* or *B*), repeatedly choosing `stay` yields reward 1 at every time step, so the return is

$$Q^*(s, \text{stay}) = 1 + \gamma + \gamma^2 + \dots = \frac{1}{1 - \gamma} = \frac{1}{1 - 0.8} = 5.$$

For `move`, the agent receives reward 0 and transitions to the other state, whose optimal value is 5, so

$$Q^*(s, \text{move}) = 0 + \gamma \cdot 5 = 0.8 \cdot 5 = 4.$$

Thus, after enough learning with ϵ -greedy exploration, the Q-values in both states converge to

$$Q(A, \text{move}) \approx 4, \quad Q(A, \text{stay}) \approx 5,$$

$$Q(B, \text{move}) \approx 4, \quad Q(B, \text{stay}) \approx 5.$$

Unlike the purely greedy case, ϵ -greedy exploration allows the agent to discover and learn the higher value of `stay`.

2.3 Bellman equation [20 points]

Without doing simulation, use Bellman equation to derive the true Q table induced by the MDP.

To compute the true optimal action-values Q^* , we use the Bellman optimality equation from lecture:

$$Q^*(s, a) = \sum_{s'} P(s' | s, a) \left[r(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right].$$

Because the system is deterministic, the transition is known for each action.

Action stay. Choosing `stay` in either state yields reward 1 and keeps the agent in the same state. Thus:

$$Q^*(s, \text{stay}) = 1 + \gamma \max_{a'} Q^*(s, a').$$

Since `stay` is the better action, the maximizer on the right-hand side is itself, so:

$$Q^*(s, \text{stay}) = 1 + \gamma Q^*(s, \text{stay}),$$

which solves to:

$$Q^*(s, \text{stay}) = \frac{1}{1 - \gamma} = \frac{1}{1 - 0.8} = 5.$$

Action move. Choosing `move` gives reward 0 and transitions to the other state s' , whose optimal value is $V^*(s') = 5$. Applying the Bellman equation:

$$Q^*(s, \text{move}) = 0 + \gamma \max_{a'} Q^*(s', a') = 0.8 \cdot 5 = 4.$$

Final Q^* table. Thus for both states *A* and *B*:

$$Q^*(\text{move}) = 4, \quad Q^*(\text{stay}) = 5.$$

These match the limiting values learned by Q-learning with exploration.